

Object-Oriented and Functional Programming Paradigms

Extended Lesson Handout with Examples, Activities, and Assessment

Course/Subject: Programming Concepts, Programming Languages, or Software Development

Target Learners: Beginner to intermediate programming students

Estimated Time: 3 to 6 hours depending on the depth of coding activities

Required Tools: Python, Java, JavaScript, PHP, or any programming language that supports OOP and functional features

1. Topic Overview

Programming paradigms are ways of thinking about how programs should be designed, organized, and written. They provide a mental model for solving problems using code. Two of the most important paradigms in modern software development are Object-Oriented Programming (OOP) and Functional Programming (FP). Both are widely used in industry, academic projects, web systems, mobile applications, data processing, artificial intelligence, and enterprise software development.

Object-Oriented Programming organizes software around objects that represent real-world entities. Each object may contain data, called attributes or properties, and behavior, called methods. Functional Programming, on the other hand, organizes software around functions and the transformation of data. It emphasizes predictable outputs, immutability, and reduced side effects. Understanding both paradigms helps students become flexible programmers who can choose the most appropriate approach for a specific problem.

This lesson compares OOP and FP not as competing ideas, but as complementary approaches. Many modern languages, such as Python, JavaScript, Scala, Kotlin, C#, and PHP, allow developers to use both styles in one project. For example, a web-based school management system may use OOP to model students, teachers, and subjects, while using functional techniques to filter, map, and summarize student records.

2. Learning Objectives

1. Define programming paradigm, object-oriented programming, and functional programming.
2. Explain the core principles of OOP, including class, object, encapsulation, inheritance, polymorphism, and abstraction.
3. Explain the core principles of FP, including pure function, immutability, first-class function, higher-order function, and recursion.
4. Compare the strengths and limitations of OOP and FP.
5. Write simple programs that demonstrate OOP and functional techniques.
6. Identify real-world situations where OOP or FP is more appropriate.
7. Appreciate the value of combining both paradigms in modern software development.

3. Introduction to Programming Paradigms

A programming paradigm is a general style or approach used to write computer programs. It influences how a programmer breaks down a problem, how data is represented, how instructions are organized, and how program behavior is controlled. Different paradigms provide different strengths. Procedural programming is useful for step-by-step tasks. Object-oriented programming is useful for modeling systems with many

interacting entities. Functional programming is useful for data transformation, mathematical computation, and predictable logic.

Learning paradigms is important because programming is not only about memorizing syntax. A student may know the correct syntax of a language but still struggle to design a maintainable program. Paradigms guide software structure. They help answer questions such as: Should the solution be divided into classes? Should the data be protected from direct access? Should the program avoid changing variables? Should a function return a new result instead of modifying an existing object?

Paradigm	Main Idea	Example Use
Procedural	Programs are written as a sequence of procedures or steps.	Simple calculations, scripts, command-line tasks
Object-Oriented	Programs are built using classes and objects.	School systems, banking systems, inventory systems
Functional	Programs are built using functions and data transformations.	Data analysis, grade computation, filtering records
Event-Driven	Program flow depends on user or system events.	Graphical interfaces, websites, mobile apps

4. Object-Oriented Programming Paradigm

Object-Oriented Programming is a paradigm that designs programs around objects. An object is a software representation of something that can have characteristics and actions. In a school system, examples of objects include Student, Teacher, Subject, Enrollment, and Grade. A student object may have attributes such as student number, name, year level, and course. It may also have methods such as enroll(), viewGrades(), and updateProfile().

OOP is especially useful when a program needs to represent a complex environment with many related entities. Instead of writing all instructions in one long file, the developer creates classes that serve as blueprints. These classes make the program easier to read, maintain, test, and extend. When a new feature is needed, the programmer can update or add a class instead of rewriting the entire system.

4.1 Class and Object

A class is a blueprint or template. It describes what data an object should have and what actions it can perform. An object is an actual instance created from the class. For example, Student is a class, while Ana, Ben, and Carlo may be individual student objects.

```
class Student:
    def __init__(self, student_id, name, course):
        self.student_id = student_id
        self.name = name
        self.course = course

    def introduce(self):
        return f"I am {self.name}, a student of {self.course}."

student1 = Student("2026-001", "Ana Cruz", "BS Computer Science")
print(student1.introduce())
```

In the example, Student is the class. The variable student1 stores an object created from that class. The object has its own student ID, name, and course. The introduce() method describes the behavior of the object.

4.2 Encapsulation

Encapsulation means keeping related data and methods together while controlling access to sensitive information. In practice, it helps prevent accidental or unauthorized changes to data. For example, a bank account balance should not be directly changed by any part of the program. Instead, it should be modified through methods such as `deposit()` and `withdraw()`, where validation rules can be applied.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount("Maria", 1000)
account.deposit(500)
print(account.get_balance())
```

Encapsulation improves security, reliability, and maintainability because it limits how data can be changed.

4.3 Inheritance

Inheritance allows one class to reuse the attributes and methods of another class. The existing class is called the parent class or superclass, while the new class is called the child class or subclass. This reduces repeated code and supports specialization.

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello, I am {self.name}."

class Teacher(Person):
    def teach(self):
        return f"{self.name} is teaching programming."

teacher1 = Teacher("Mr. Santos")
print(teacher1.greet())
print(teacher1.teach())
```

In the example, `Teacher` inherits from `Person`. The `Teacher` class can use the `greet()` method from `Person` and can also define its own method, `teach()`.

4.4 Polymorphism

Polymorphism means “many forms.” It allows the same method name or operation to behave differently depending on the object using it. This is useful when different objects share a common action but perform it in different ways.

```
class Dog:
    def speak(self):
        return "Bark"

class Cat:
    def speak(self):
        return "Meow"
```

```
animals = [Dog(), Cat()]
for animal in animals:
    print(animal.speak())
```

Both Dog and Cat have a speak() method, but each object responds differently. Polymorphism supports flexible and reusable code.

4.5 Abstraction

Abstraction means showing only the important details and hiding unnecessary complexity. For example, when a user clicks a “Submit” button in a system, the user does not need to know the database queries, validations, and server processes behind it. The system exposes a simple action while hiding the internal process.

In programming, abstraction helps developers manage complexity. A class can provide simple methods that represent high-level operations, while the technical details remain inside the class.

4.6 Advantages and Limitations of OOP

- Advantages: OOP promotes code reusability, modularity, maintainability, and real-world modeling.
- Advantages: It is effective for large systems with many entities and relationships.
- Limitations: It can be more complex for beginners because it requires understanding classes, objects, and relationships.
- Limitations: Poorly designed inheritance can make systems difficult to modify.
- Limitations: It may not be the simplest approach for small mathematical or data-processing tasks.

5. Functional Programming Paradigm

Functional Programming is a paradigm that treats computation as the evaluation of functions. Instead of focusing on objects that change state, FP focuses on transforming input data into output data. A functional programmer often asks: What is the input? What transformation should be applied? What output should be returned?

FP encourages the use of small, reusable, and predictable functions. These functions are easier to test because they usually depend only on their inputs. When a function does not change external data, it produces fewer unexpected effects. This is especially useful in data analysis, artificial intelligence, concurrent programming, financial computation, and systems where correctness is important.

5.1 Pure Functions

A pure function always produces the same output for the same input and does not change anything outside the function. This makes the function predictable and easier to test.

```
def add_tax(price):
    return price * 1.12

print(add_tax(100)) # always returns 112.0
```

The add_tax() function is pure because it only depends on the price input and returns a result. It does not modify a global variable, change a file, or update a database.

5.2 Immutability

Immutability means that data should not be changed after it is created. Instead of modifying an existing value, a program creates a new value. This reduces accidental changes and makes data flow easier to understand.

```
grades = [85, 90, 92]
new_grades = grades + [88]
```

```
print(grades)      # original list remains [85, 90, 92]
print(new_grades)  # new list becomes [85, 90, 92, 88]
```

5.3 First-Class Functions

In languages that support first-class functions, functions can be assigned to variables, passed as arguments, and returned from other functions. This allows flexible program design.

```
def greet(name):
    return f"Good day, {name}!"

message_function = greet
print(message_function("Alex"))
```

5.4 Higher-Order Functions

A higher-order function is a function that receives another function as an argument or returns a function. Common examples include `map()`, `filter()`, and `reduce()`. These are useful for processing collections of data.

```
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda n: n * n, numbers))
even_numbers = list(filter(lambda n: n % 2 == 0, numbers))

print(squares)
print(even_numbers)
```

The `map()` function transforms every item in a collection, while `filter()` selects items that satisfy a condition.

5.5 Recursion

Recursion is a technique where a function calls itself to solve a smaller version of the same problem. In functional programming, recursion is often used instead of loops.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(5))  # 120
```

The factorial function solves $5!$ by computing $5 \times 4 \times 3 \times 2 \times 1$. The base case stops the recursion when n becomes 0.

5.6 Advantages and Limitations of FP

- Advantages: FP encourages clean, predictable, and testable functions.
- Advantages: It reduces unexpected side effects by discouraging direct changes to shared data.
- Advantages: It works well for data transformation, filtering, mapping, and statistical processing.
- Limitations: It may be difficult for beginners who are used to step-by-step procedural logic.
- Limitations: Recursive solutions can be harder to understand at first and may cause performance issues if not optimized.
- Limitations: Some real-world systems are easier to model with objects than with functions alone.

6. Comparison Between OOP and Functional Programming

Both OOP and FP can solve many of the same problems, but they approach design differently. OOP focuses on modeling entities and their interactions. FP focuses on transforming data using functions. The best choice depends on the problem, language, team skill, and system requirements.

Aspect	Object-Oriented Programming	Functional Programming
--------	-----------------------------	------------------------

Main focus	Objects and classes	Functions and data transformation
Data handling	Data may change through object methods	Data is preferably immutable
Basic unit	Object	Function
Reuse strategy	Inheritance, composition, interfaces	Reusable pure functions and higher-order functions
Best for	Large systems with many entities	Data processing, calculations, and predictable logic
Common concepts	Class, object, inheritance, encapsulation	Pure function, map, filter, recursion
Risk	Overly complex class hierarchies	Hard-to-read nested functions or recursion
Example problem	Library management system	Computing final grades from score lists

7. Using OOP and FP Together

Modern programming often combines multiple paradigms. A developer may use OOP to structure a system and FP to process data inside that system. This mixed approach is practical because it allows the program to benefit from organized objects and predictable functions.

Consider a student grading system. The system may use a `Student` class to represent each learner. At the same time, it may use functional techniques to compute averages, filter passing students, and sort grades. This combination produces readable and maintainable code.

```
class Student:
    def __init__(self, name, grades):
        self.name = name
        self.grades = grades

    def average(self):
        return sum(self.grades) / len(self.grades)

students = [
    Student("Ana", [90, 88, 95]),
    Student("Ben", [70, 75, 72]),
    Student("Carlo", [85, 89, 91])
]

passing_students = list(filter(lambda s: s.average() >= 75, students))
for student in passing_students:
    print(student.name, student.average())
```

The `Student` class shows OOP because it models a real-world entity. The `filter()` function shows FP because it processes a collection using a function.

8. Real-World Applications

- **School Information System:** OOP can model students, teachers, subjects, schedules, and enrollment records. FP can compute statistics such as average grades, pass rates, and filtered honor lists.
- **Banking System:** OOP can model accounts, customers, loans, and transactions. FP can compute interest, service charges, and financial summaries using pure functions.

- **Disaster Reporting Platform:** OOP can model users, incident reports, responders, evacuation centers, and alerts. FP can process reports, filter emergency levels, and summarize incident counts per barangay.
- **E-Commerce Application:** OOP can model products, customers, carts, and orders. FP can calculate discounts, totals, and filtered product lists.
- **Machine Learning and Data Processing:** OOP can organize models, datasets, and training modules. FP can transform datasets through mapping, filtering, normalization, and statistical computation.

9. Suggested Lesson Delivery Plan

Part	Time	Teacher Activity	Student Activity
Motivation	15 minutes	Ask students how they would model a school system.	Share examples of real-world objects.
OOP Discussion	60 minutes	Explain class, object, and OOP principles with examples.	Create simple classes and objects.
FP Discussion	60 minutes	Explain pure functions, immutability, map, filter, and recursion.	Write small functions and process lists.
Comparison	30 minutes	Discuss when to use each paradigm.	Classify sample problems as OOP, FP, or mixed.
Hands-on Coding	90 minutes	Guide students in creating a mini grade-processing program.	Build and test the program.
Assessment	30 minutes	Administer quiz and reflection activity.	Answer questions and submit outputs.

10. Learning Activities

Activity 1: Identify the Paradigm

Direction: Identify whether the following situations are best solved using OOP, FP, or a combination of both.

8. A system that manages students, teachers, classrooms, and subjects.
9. A program that converts a list of temperatures from Celsius to Fahrenheit.
10. A banking application with customers, accounts, and transactions.
11. A program that filters all failing grades from a list of student scores.
12. A web application that stores user profiles and summarizes user feedback.

Suggested Answers: 1. OOP; 2. FP; 3. OOP; 4. FP; 5. Combination of OOP and FP.

Activity 2: Create a Class

Direction: Create a class named Book with the following attributes: title, author, and year. Add a method named display_info() that prints the book details.

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year
```

```
def display_info(self):
    return f"{self.title} by {self.author}, published in {self.year}."
```

Activity 3: Use Functional Techniques

Direction: Given a list of numbers, use map() to square each number and filter() to get only even numbers.

```
numbers = [1, 2, 3, 4, 5, 6]
squared = list(map(lambda x: x * x, numbers))
even = list(filter(lambda x: x % 2 == 0, numbers))
print(squared)
print(even)
```

Activity 4: Mini Project

Create a simple student grade analyzer. Use OOP to represent each student and FP to process the list of students. The program should display each student average, identify passing students, and compute the class average.

- Create a Student class with name and grades.
- Add a method that computes the average grade.
- Create at least five student objects.
- Use filter() to display students with an average of 75 or higher.
- Use map() or list comprehension to get all averages.
- Compute and display the overall class average.

11. Assessment

A. Identification

13. A blueprint for creating objects. Answer: Class
14. An instance of a class. Answer: Object
15. The OOP principle that hides internal data and controls access. Answer: Encapsulation
16. A function that always returns the same result for the same input and has no side effects. Answer: Pure Function
17. A function that accepts another function as an argument or returns a function. Answer: Higher-Order Function

B. Essay Questions

18. Explain why OOP is useful in developing a school information system.
19. Explain why pure functions are easier to test than functions that change global variables.
20. Compare inheritance and higher-order functions as two different ways of promoting code reuse.
21. Give a real-world system where OOP and FP can be used together. Explain your answer.

C. Coding Task

Create a simple program for an inventory system. Use OOP to model Product objects. Each product should have a name, price, and quantity. Then use functional techniques to compute the total value of all products, filter low-stock items, and display the product names.

12. Rubric for Mini Project

Criteria	Excellent (5)	Satisfactory (3)	Needs Improvement (1)
OOP Design	Classes and objects are correctly used.	Classes are present but incomplete.	No clear class/object design.
Functional Techniques	Correct use of map/filter/list	Functional techniques are	No functional technique

	transformations.	partially correct.	used.
Program Correctness	Output is accurate and complete.	Output has minor errors.	Output is incomplete or incorrect.
Code Readability	Code is clean, organized, and easy to understand.	Code is understandable but needs improvement.	Code is difficult to read.
Explanation	Student clearly explains the paradigm used.	Student gives a limited explanation.	Student cannot explain the solution.

13. Summary

Object-Oriented Programming and Functional Programming are two essential paradigms in computer programming. OOP focuses on classes and objects, making it suitable for modeling real-world systems with interacting components. Its key concepts include class, object, encapsulation, inheritance, polymorphism, and abstraction. FP focuses on functions and data transformation, making it suitable for predictable computation and data processing. Its key concepts include pure function, immutability, first-class function, higher-order function, and recursion.

A skilled programmer should not treat these paradigms as enemies. Instead, the programmer should understand the strengths of each paradigm and apply them appropriately. In many modern applications, OOP provides structure while FP provides clean and reliable data processing. Mastering both paradigms helps students become better problem solvers and more adaptable software developers.

14. Glossary of Important Terms

- **Abstraction:** The process of hiding unnecessary details and showing only essential features.
- **Class:** A blueprint for creating objects.
- **Encapsulation:** The bundling of data and methods while controlling access to data.
- **First-Class Function:** A function that can be assigned, passed, or returned like a value.
- **Functional Programming:** A paradigm that focuses on functions and data transformations.
- **Higher-Order Function:** A function that accepts another function or returns a function.
- **Immutability:** The principle of not changing data after it is created.
- **Inheritance:** The ability of one class to acquire the properties and methods of another class.
- **Object:** An instance of a class.
- **Object-Oriented Programming:** A paradigm that focuses on objects and classes.
- **Polymorphism:** The ability of the same method or operation to behave differently depending on the object.
- **Pure Function:** A function that has no side effects and returns the same output for the same input.
- **Recursion:** A technique where a function calls itself.

15. Suggested References and Learning Resources

22. Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and Interpretation of Computer Programs. MIT Press.
23. Bloch, J. (2018). Effective Java. Addison-Wesley.
24. Downey, A. (2015). Think Python: How to Think Like a Computer Scientist. O'Reilly Media.
25. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
26. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
27. Van Roy, P., & Haridi, S. (2004). Concepts, Techniques, and Models of Computer Programming. MIT Press.